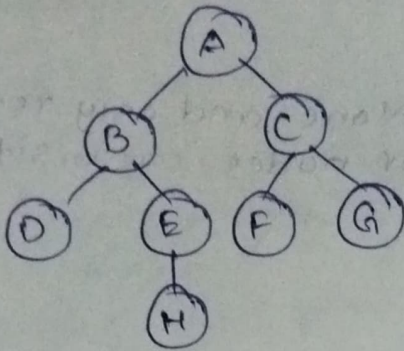


Lab-2b: Depth-First Search

Given tree,



path to find,

$A \rightarrow H$.

DFS: Go as deep as possible

$A \rightarrow B \rightarrow D$

$B \rightarrow E \rightarrow H$

$A \rightarrow C \rightarrow F$

$C \rightarrow G$.

Not level-by-level
but as deep as possible
and then coming back to
go to another side.

→ DFS uses stack data structure which follows
LIFO → last in first out.
so that, we can go deep as possible while traversing
the nodes.

- we maintain visited list here also, so that we
will not go to the same node again,

- This process continues until the stack is not
empty or element is found.

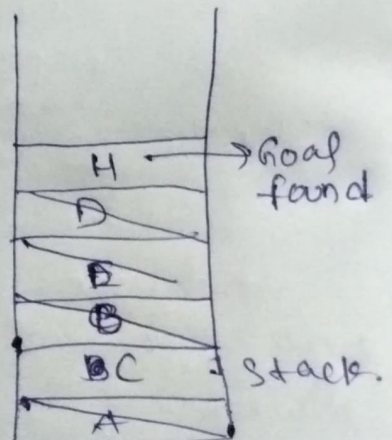
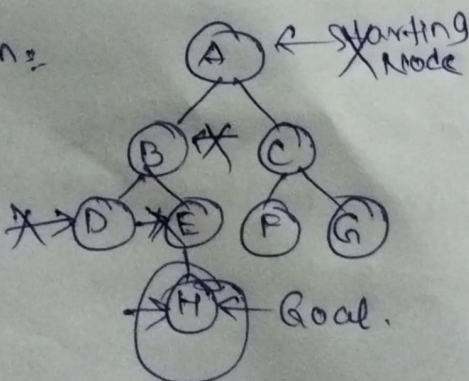
- The process is same as BFS,

if node found → return path & visited

if node already not visited → add that to

visited and add it's corresponding
edges to the stack, ~~and~~ ~~path~~ in reverse order.

Dry run:



visited:

A	B	D	E	
---	---	---	---	--

path: $A \rightarrow B \rightarrow E \rightarrow H$

visited: $A \rightarrow B \rightarrow D \rightarrow E$

- Is DFS better than BFS?

No. Because, if the goal is on right side here, all the left node traversals are not necessary.
- so, DFS visits so many unnecessary nodes as BFS visits the nodes level-by-level, it reduces the visits of unnecessary nodes.